This span_or query fetches the document with Newton's quote because it matches both Plato and Aristotle. Note that the friends query is not a match, but because the operator is OR, the query is happy to proceed because at least one of the words queried was a match. The query does not fail, although the word *friends* is not a match.

Elastic search has other span queries such as span_not, span_containing, span_multi_term, and others, but unfortunately, we cannot discuss all of these types here. I advise you to consult the documentation to better understand these queries. The documentation is available here: http://mng.bz/QPa4.

The next section deals with specialized queries such as distance_feature, percolator, and others. Let's turn our attention to those now.

12.9 Specialized queries

In addition to the types of queries we have seen so far, Elasticsearch has several advanced queries dedicated to serving specialized functions: for example, boosting the score for cafes serving chilled drinks at a specified location (distance_feature query), alerting the user when an item is back in stock (percolate query), finding similar-looking documents (more_like_this query), giving documents more importance (pinned query), and so on. We dedicate the final section of this chapter to looking at these specialized queries in detail.

12.9.1 The distance_feature query

When searching for classic literature, suppose we want to add a clause to find books published in 1813. Along with returning all books that are literature classics, we expect to find *Pride and Prejudice* (Jane Austen's classic), but we want to put *Pride and Prejudice* at the top of the list because it was printed in 1813. Topping the list means boosting the relevance score of the query results based on a particular clause; in this case, we specifically want books published in 1813 to be given higher importance. This functionality is available in Elasticsearch by using the distance_feature query. The query fetches results and marks them with a higher relevancy score if they are nearer to an origin date (1813, in this example).

The distance_feature query provides similar support for locations. We can highlight the locations nearer a particular address and boost them to the top of the list if we desire. Say that we want to find all restaurants serving fish and chips, but those topping the list should be near Borough Market by London Bridge. (Borough Market is a world-renowned thirteenth-century artisan food market; see https://boroughmarket.org.uk.)

We can use the distance_feature query for such use cases, to find results near an origin location or date. The dates and locations are fields declared as date (or date_nanos) and geo_point data types, respectively. Results closer to the given date or location are rated higher in relevance scores. Let's look at some examples to understand the concept in detail.